🦫 제1특강: 비버 챌린지 및 컴퓨팅 퍼즐 집중 공략

> **"복잡한 프로그래밍 코드가 아닙니다! 규칙을 읽고 손으로 쫓아가는 직관과 디버깅 능력이 100점을 만듭니다."**

비버 챌린지(Bebras Challenge)는 전 세계 수십 개국 청소년들이 참가하는 국제 정보(Informatics) 사고력 대회입니다. 디미고 소질·적성검사의 컴퓨팅 사고력 및 융합형 문제들은 이 비버 챌린지의 문제 출제 패턴과 매우 밀접하게 닿아 있습니다.

이 특강에서는 디미고 시험에서 가장 변별력이 높게 출제되는 비버 챌린지형 4대 핵심 퍼즐 유형을 정리하고, 시험장에서 1초의 망설임 없이 정답을 찾아내는 손풀이 비법을 전수합니다.

---

1. 유한 상태 머신 (Finite State Machine)과 화살표 추적

컴퓨터 시스템이나 기계 장치는 **'현재 상태'**에서 외부의 **'입력'**을 받아 정해진 규칙(화살표)에 따라 **'다음 상태'**로 변화합니다. 이를 **상태 머신(State Machine)**이라고 부릅니다.

💡 일상 속의 상태 머신: 지하철 개찰구

- **상태 1 [잠김(Locked)]**: 사람이 밀어도 돌아가지 않음.
- **입력 [교통카드 태그]** 발생 ➔ **상태 2 [열림(Unlocked)]**로 전이!
- **입력 [사람 통과]** 발생 ➔ 다시 **상태 1 [잠김(Locked)]**으로 전이!



🎯 시험장 100% 득점 행동 강령

1. **절대 머리로만 화살표를 따라가지 마세요.**
2. 시험지 여백의 그림에서 **현재 상태의 동그라미에 연필을 올려놓고**, 입력 신호가 들어올 때마다 화살표를 따라 연필 끝을 이동하세요.
3. 중간에 상태가 바뀔 때마다 **상태 이름과 출력값을 순서대로 시험지에 적는 습관(Trace)**이 실수를 0으로 만듭니다.

---

2. 그리디(탐욕) 최적화와 스케줄링 퍼즐 (Greedy & Activity Selection)

비버 챌린지와 디미고 적성검사에서 출제되는 '자원 최적화' 퍼즐의 단골 유형입니다. 시간이 겹치는 여러 활동(동아리 체험, 강연, 회의 등) 중에서 **"겹치지 않게 가장 많은 활동을 선택하는 전략"**을 묻습니다.

💡 핵심 행동 강령: "종료 시간(끝나는 시간)이 가장 빠른 것부터 골라라!"

- **왜 시작 시간이 아니라 종료 시간일까?**
  - 어떤 활동을 가능한 한 일찍 끝내야, **그 뒤에 다른 활동을 선택할 수 있는 '남은 시간'이 가장 넓게 확보**되기 때문입니다.
- **그리디(Greedy) 선택 3단계 풀이법**:
  1. **1단계: 정렬 ➔** 모든 활동을 **'종료 시간'이 빠른 순서**대로 시험지 여백에 나열합니다.
  2. **2단계: 첫 번째 선택 ➔** 가장 일찍 끝나는 활동을 무조건 첫 번째로 선택합니다.
  3. **3단계: 겹치지 않는 것 선택 ➔** 직전 활동이 끝난 시간 이후에 시작하는 활동 중에서, 그다음으로 가장 일찍 끝나는 것을 골라나갑니다. 이미 시간이 지난 활동은 가차 없이 X표를 칩니다.

---

3. 데이터 표현과 흑백 도트 압축 (런-랭스 인코딩, RLE)

컴퓨터가 그림이나 문자의 파일 용량을 줄이기 위해 사용하는 대표적인 무손실 압축 기술입니다. 비버 챌린지에서는 **격자 픽셀 그림을 숫자로 변환하는 문제**로 100% 출제됩니다.

💡 런-랭스 인코딩(Run-Length Encoding)의 기본 원리

동일한 데이터가 연속해서 나타나는 길이(Run Length)를 묶어서 **'반복되는 값 + 반복 횟수'**로 표현합니다.

- **원본 데이터**: `W W W W W B B B W W B` (총 11글자)
  - 흰색(W)이 연속 5개 ➔ `W5`
  - 검은색(B)이 연속 3개 ➔ `B3`
  - 흰색(W)이 연속 2개 ➔ `W2`
  - 검은색(B)이 연속 1개 ➔ `B1`
- **압축 결과**: `W5 B3 W2 B1` (8글자로 단축!)

⚠️ 비버 챌린지 시험 문제의 단골 함정

문제마다 **"항상 흰색(W)부터 시작하여 개수를 적는다"**라는 특수 규칙이 주어질 때가 많습니다.

- 만약 첫 번째 픽셀이 검은색(B)으로 시작한다면?
  - 흰색이 0개 있는 것이므로, 맨 앞에 **0**을 먼저 적어주고 시작해야 합니다!
  - 예: `B B B W W` ➔ `0 3 2` (흰색 0개, 검은색 3개, 흰색 2개)
- **시험 팁**: 첫 시작 규칙(0부터 시작하는지, 색깔 기호가 붙는지)을 문제 지문에서 반드시 확인하세요!

4. 정보 보안과 규칙 암호 해독

암호 문제는 고난도 수학이 아니라, 일정한 **규칙(알고리즘)에 따라 글자를 변환하는 패턴**을 찾는 퍼즐입니다.

💡 카이사르(시저) 암호 (Caesar Cipher)

- **원리**: 알파벳을 정해진 숫자(키 값)만큼 오른쪽으로 밀어서 표기합니다.
- 예: 키 값이 **+3**인 경우
  - A ➔ D, B ➔ E, C ➔ F, ..., X ➔ A, Y ➔ B, Z ➔ C
- **복호화(원문 찾기)**: 거꾸로 키 값만큼 **왼쪽(-3)**으로 되돌립니다.

🎯 암호 문제 10초 단축 꿀팁

암호 문제가 보이면 즉시 시험지 여백에 알파벳 26자를 빠르게 적어두세요.

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

눈으로 알파벳 순서를 떠올리다가는 반드시 Z ➔ A로 넘어가는 순환 부분에서 1칸 오차가 발생합니다. 적어둔 표를 손가락으로 짚어가며 풀면 100% 정답을 맞힙니다.

---

5. 실전 킬러 연습문제

[문제 1] 비버의 자동 음료 자판기 (상태 머신 추적)

어느 자판기는 현재 **[대기 상태]**에 있습니다. 이 자판기의 동작 규칙은 다음과 같습니다.

- **[대기 상태]**에서:
  - 100원 동전을 넣으면 ➔ **[100원 상태]**로 전이합니다.
  - 500원 동전을 넣으면 ➔ **[500원 상태]**로 전이합니다.
- **[100원 상태]**에서:
  - 100원 동전을 넣으면 ➔ **[200원 상태]**로 전이합니다.
  - 500원 동전을 넣으면 ➔ **[반환 상태]**로 전이하고 모든 돈을 뱉어낸 뒤 **[대기 상태]**로 복귀합니다.
- **[200원 상태]**에서:
  - '음료 버튼'을 누르면 ➔ 음료수가 1캔 나오고 즉시 **[대기 상태]**로 복귀합니다.
  - 100원 동전을 넣으면 ➔ 추가 투입을 무시하고 그대로 **[200원 상태]**를 유지합니다.
- **[500원 상태]**에서:
  - '음료 버튼'을 누르면 ➔ 음료수가 1캔 나오고 잔돈 300원을 배출한 뒤 **[대기 상태]**로 복귀합니다.

처음에 [대기 상태]인 자판기에 아래와 같은 순서로 입력이 들어왔을 때, 최종적으로 배출된 **음료수의 총 개수**와 자판기 안에 **남아있는 잔액**은 얼마일까요?

> **입력 순서**: [100원] ➔ [100원] ➔ [음료 버튼] ➔ [100원] ➔ [500원] ➔ [500원] ➔ [음료 버튼]

[해설]
차근차근 상태 전이 과정을 손으로 추적합니다.

1. 시작: [대기 상태] (잔액 0원)
2. [100원 투입]: [100원 상태]
3. [100원 투입]: [200원 상태]
4. [음료 버튼]: 음료수 1캔 배출 ➔ [대기 상태] 복귀 (음료 1캔, 잔액 0원)
5. [100원 투입]: [100원 상태]
6. [500원 투입]: 규칙에 따라 [반환 상태]로 전이하여 600원 전액 반환 ➔ [대기 상태] 복귀 (음료 1캔, 잔액 0원)
7. [500원 투입]: [500원 상태]
8. [음료 버튼]: 음료수 1캔 배출 + 잔돈 300원 배출 ➔ [대기 상태] 복귀 (음료 누적 2캔, 잔액 0원)

[정답] **음료수: 2캔, 남은 잔액: 0원**

[문제 2] 비버의 축제 체험 부스 스케줄링 (그리디 최적화)

디미고 축제에서 비버가 다음과 같이 시간표가 짜인 6개의 인기 IT 체험 부스(A~F)에 참여하려고 합니다. 각 부스는 시작 시간부터 종료 시간까지 한 번 들어가면 중간에 나올 수 없으며, 두 개 이상의 부스를 동시에 체험할 수는 없습니다. 비버가 하루 동안 참여할 수 있는 **체험 부스의 최대 개수**와 **선택한 부스들의 조합**은 무엇일까요?

- **부스 A**: 10:00 ~ 11:30
- **부스 B**: 10:30 ~ 12:00
- **부스 C**: 11:30 ~ 13:00
- **부스 D**: 12:00 ~ 14:00
- **부스 E**: 13:00 ~ 14:30
- **부스 F**: 14:00 ~ 15:30

[해설]
겹치지 않게 가장 많은 활동을 선택하려면 **"종료 시간이 가장 빠른 것부터 선택하는 그리디 전략"**을 적용합니다.

1. **종료 시간 기준 오름차순 정렬**:
    - 1순위: 부스 A (11:30 종료)
    - 2순위: 부스 B (12:00 종료)

    - 3순위: 부스 C (13:00 종료)
    - 4순위: 부스 D (14:00 종료)
    - 5순위: 부스 E (14:30 종료)
    - 6순위: 부스 F (15:30 종료)
2. **순차적 선택 진행**:
    - ➔ 가장 먼저 끝나는 **부스 A (10:00 ~ 11:30)** 선택! (현재 시각: 11:30)
    - ➔ 부스 B는 10:30에 시작하므로 A와 겹쳐서 탈락!
    - ➔ 그다음으로 빠른 **부스 C (11:30 ~ 13:00)** 선택! (A가 끝난 11:30에 바로 시작 가능, 현재 시각: 13:00)
    - ➔ 부스 D는 12:00에 시작하므로 C와 겹쳐서 탈락!
    - ➔ 그다음으로 빠른 **부스 E (13:00 ~ 14:30)** 선택! (C가 끝난 13:00에 바로 시작 가능, 현재 시각: 14:30)
    - ➔ 부스 F는 14:00에 시작하므로 E와 겹쳐서 탈락!
3. 따라서 겹치지 않고 최대로 참여할 수 있는 부스의 수는 **3개 (A ➔ C ➔ E)**입니다.

[정답] **최대 3개 (부스 A, C, E)**

[문제 3] 흑백 픽셀 아이콘의 RLE 복원

어느 컴퓨터 화면에 가로 5픽셀, 세로 5픽셀(총 25픽셀) 크기의 흑백 아이콘이 있습니다. 이 아이콘은 왼쪽 위 첫 번째 픽셀부터 오른쪽으로 한 줄씩 읽어가며 런-랭스 인코딩(RLE)으로 압축되었습니다.

- **압축 규칙**: 항상 흰색(□)의 연속 개수부터 시작하여 흰색 개수와 검은색(■) 개수를 번갈아 가며 숫자로 기록합니다. (만약 첫 픽셀이 검은색이면 맨 앞 숫자는 0이 됩니다.)
- **압축된 데이터**: `2, 1, 4, 1, 4, 1, 4, 1, 7`

이 압축 데이터를 5×5 격자에 복원했을 때, **정중앙 위치(3번째 줄의 3번째 칸)**의 픽셀 색상은 무엇일까요?

[해설]
숫자가 의미하는 색상과 개수를 순서대로 나열합니다. (총합 = 2 + 1 + 4 + 1 + 4 + 1 + 4 + 1 + 7 = 25 픽셀)

- 흰색 2개 (1~2번 픽셀)
- 검은색 1개 (3번 픽셀)
- 흰색 4개 (4~7번 픽셀)
- 검은색 1개 (8번 픽셀)
- 흰색 4개 (9~12번 픽셀)
- 검은색 1개 (13번 픽셀)
- 흰색 4개 (14~17번 픽셀)
- 검은색 1개 (18번 픽셀)

- 흰색 7개 (19~25번 픽셀)

5×5 격자에서 정중앙 칸은 **3번째 줄의 3번째 칸**이므로, 전체 번호로는 $(3-1)\times 5+3=13$번째 픽셀입니다.

- 13번째 픽셀의 색상을 확인해 보면, 바로 **검은색(■)**에 해당합니다!

[정답] **검은색 (또는 ■)**

[문제 4] 디미고 비밀 암호 해독 (카이사르 복호화)

디미고 동아리 선배가 신입생들에게 아래와 같은 암호문과 암호화 규칙을 전달했습니다.

- **암호문**: `GLPLJR`
- **암호화 규칙**: 원문의 각 알파벳을 일정한 글자 수 *K*만큼 뒤로(오른쪽으로) 밀어서 작성했습니다.
- **추가 단서**: 암호문의 첫 글자 `G`의 원래 알파벳은 `D`였습니다.

이 암호문의 원래 단어(원문)는 무엇일까요?

[해설]

1. 단서 분석: 원문 `D`가 암호문 `G`로 바뀌었습니다.
    - D(4번째) ➔ E(1칸) ➔ F(2칸) ➔ G(3칸)
    - 즉, 오른쪽으로 **+3칸** 밀어낸 카이사르 암호입니다 ($K=3$).
2. 복호화 규칙: 원문을 찾으려면 각 암호문 글자를 왼쪽으로 **-3칸** 되돌려야 합니다.
    - `G` - 3 ➔ **D**
    - `L` - 3 ➔ **I** (I ➔ J ➔ K ➔ L)
    - `P` - 3 ➔ **M** (M ➔ N ➔ O ➔ P)
    - `I` - 3 ➔ **F** (F ➔ G ➔ H ➔ I)
    - `J` - 3 ➔ **G** (G ➔ H ➔ I ➔ J)
    - `R` - 3 ➔ **O** (O ➔ P ➔ Q ➔ R)
3. 원문 조합: **DIMIGO** (디미고!)

[정답] **DIMIGO**

📊 제2특강: 정렬 및 탐색 알고리즘 손추적 특강

> **"코딩 시험이 아닙니다! 중간 과정을 시험지 여백에 한 줄씩 적어 내려가는 '손추적(Hand-Tracing)'이 실수를 0으로 만듭니다."**

디미고 소질·적성검사의 컴퓨팅 사고력 영역에서 '정렬(Sorting)'과 '탐색(Search)'은 거의 매년 1~2문제씩 고정 출제되는 핵심 주제입니다. 시험에서는 학생들에게 복잡한 프로그래밍 코드를 작성하라고 요구하지 않습니다. 대신 **"알고리즘의 규칙에 따라 단계별로 데이터가 어떻게 변하는지 그 과정을 쫓아갈 수 있는가?"**를 평가합니다.

이 특강에서는 버블 정렬, 선택 정렬, 삽입 정렬의 회전별 상태 변화와 시험 단골 킬러 문제인 '최소 교환 횟수 구하기', '이진 탐색 횟수 계산'을 완벽하게 마스터합니다.

---

1. 왜 눈으로 풀면 100% 틀릴까? (손추적의 원칙)

정렬 문제는 머릿속으로만 생각하면 앞뒤 숫자가 꼬여서 반드시 실수가 발생합니다.

- **만점자의 풀이 습관**: 시험지 빈 공간에 대괄호 `[ ]`를 그리고, 한 회전(Pass)이 끝날 때마다 숫자가 바뀐 상태를 **한 줄씩 직접 쓰면서** 풉니다.
- 회전이 끝날 때마다 **자리가 확정된 숫자에는 밑줄이나 동그라미**를 쳐두면 다음 회전에서 비교 대상에서 제외하기가 매우 쉽습니다.

---

2. 버블 정렬 (Bubble Sort) 회전별 손추적

> **"이웃한 두 숫자를 비교해서 큰 놈을 오른쪽으로 밀어낸다!"**

거품이 수면 위로 뽀글뽀글 올라오는 것처럼, 매 회전마다 **남은 숫자들 중 가장 큰 숫자가 맨 오른쪽 끝자리로** 이동하여 확정됩니다.

💡 실전 추적 예제

배열: `[7, 2, 5, 3, 1]` (5개의 숫자를 오름차순으로 정렬)

- **[1회전 (Pass 1)]**:
  - (7, 2) 비교 ➔ 7이 크므로 교환: `[2, 7, 5, 3, 1]`
  - (7, 5) 비교 ➔ 7이 크므로 교환: `[2, 5, 7, 3, 1]`

    - (7, 3) 비교 ➔ 7이 크므로 교환: `[2, 5, 3, 7, 1]`
    - (7, 1) 비교 ➔ 7이 크므로 교환: `[2, 5, 3, 1, 7]`
    - 👉 **1회전 결과**: `[2, 5, 3, 1, 7]` (가장 큰 **7**이 맨 뒤에 고정 확정!)
- **[2회전 (Pass 2)]**: (확정된 7은 제외하고 앞의 4개만 비교)
    - (2, 5) 비교 ➔ 2가 작으므로 그대로: `[2, 5, 3, 1, 7]`
    - (5, 3) 비교 ➔ 5가 크므로 교환: `[2, 3, 5, 1, 7]`
    - (5, 1) 비교 ➔ 5가 크므로 교환: `[2, 3, 1, 5, 7]`
    - 👉 **2회전 결과**: `[2, 3, 1, 5, 7]` (두 번째로 큰 **5** 고정 확정!)
- **[3회전 (Pass 3)]**: (5와 7을 제외한 앞의 3개만 비교)
    - (2, 3) 비교 ➔ 그대로: `[2, 3, 1, 5, 7]`
    - (3, 1) 비교 ➔ 3이 크므로 교환: `[2, 1, 3, 5, 7]`
    - 👉 **3회전 결과**: `[2, 1, 3, 5, 7]` (**3** 고정 확정!)
- **[4회전 (Pass 4)]**: (앞의 2개만 비교)
    - (2, 1) 비교 ➔ 2가 크므로 교환: `[1, 2, 3, 5, 7]`
    - 👉 **최종 정렬 완료**: `[1, 2, 3, 5, 7]`

> ⚠️ **시험 단골 질문**: "버블 정렬 2회전이 끝난 직후의 배열 상태는?" ➔ 정답: `[2, 3, 1, 5, 7]`

---

3. 선택 정렬 (Selection Sort) 회전별 손추적

> **"남은 숫자들 중에서 제일 작은 놈을 딱 찍어서(선택) 맨 앞자리와 맞바꾼다!"**

매 회전마다 **남은 숫자들 중 가장 작은 숫자(최솟값)를 찾아 맨 왼쪽 자리로** 데려와 확정합니다.

💡 실전 추적 예제

배열: `[7, 2, 5, 3, 1]` (동일한 배열을 선택 정렬로 정렬)

- **[1회전 (Pass 1)]**:
    - 전체 `[7, 2, 5, 3, 1]` 중 최솟값은 **1**입니다.
    - 최솟값 **1**과 맨 첫 번째 자리의 **7**을 맞바꿉니다(Swap).
    - 👉 **1회전 결과**: `[1, 2, 5, 3, 7]` (가장 작은 **1**이 맨 앞에 고정 확정!)
- **[2회전 (Pass 2)]**: (확정된 1을 제외한 `[2, 5, 3, 7]` 중 탐색)
    - 남은 것 중 최솟값은 **2**입니다.
    - 2는 이미 두 번째 자리에 있으므로 자기 자신과 교환(사실상 이동 없음).

    - 👉 **2회전 결과**: `[1, 2, 5, 3, 7]` (**2** 고정 확정!)
- **[3회전 (Pass 3)]**: (확정된 1, 2를 제외한 `[5, 3, 7]` 중 탐색)
    - 남은 것 중 최솟값은 **3**입니다.
    - 최솟값 **3**과 세 번째 자리의 **5**를 맞바꿉니다.
    - 👉 **3회전 결과**: `[1, 2, 3, 5, 7]` (**3** 고정 확정!)
- **[4회전 (Pass 4)]**: (남은 `[5, 7]` 중 탐색)
    - 최솟값 **5**는 이미 네 번째 자리에 있으므로 유지.
    - 👉 **최종 정렬 완료**: `[1, 2, 3, 5, 7]`

---

4. 삽입 정렬 (Insertion Sort) 직관적 이해

> **"손안에 든 카드를 정리하듯, 새 카드를 이미 정렬된 부분의 알맞은 틈새에 쏙 끼워 넣는다!"**

- **동작 원리**:
    - 1번째 원소는 이미 정렬되어 있다고 봅니다.
    - 2번째 원소부터 시작하여, 앞쪽에 있는 숫자들과 비교하면서 자기보다 큰 숫자들은 뒤로 밀어내고 제자리를 찾아 들어갑니다.
- **장점**: 데이터가 이미 어느 정도 정렬되어 있는 상태라면 비교와 이동을 거의 하지 않아 매우 빠르게 끝납니다.

---

5. 카드 최소 맞바꾸기(Swap) 퀴즈 공략법

디미고 시험에서 가장 매력적인 킬러 퍼즐 중 하나는 **"알고리즘 제한 없이, 사람이 눈으로 보고 두 카드를 맞바꿀 때 최소 몇 번 만에 오름차순으로 만들 수 있는가?"**를 묻는 문제입니다.

💡 사이클(Cycle) 분석 기법 (만점 비법)

카드를 제자리로 보내는 관계를 화살표로 연결해 보면 닫힌 원(사이클)이 만들어집니다.

- **공식**: **(최소 교환 횟수) = (제자리가 아닌 카드의 수) - (독립된 사이클의 수)**
- 더 직관적인 방법: **"현재 카드를 자기가 원래 가야 할 자리의 카드와 1:1로 맞바꿔나가는 과정"**을 직접 손으로 시뮬레이션합니다.

---

6. 업다운(Up-Down) 이진 탐색 (Binary Search)과 횟수 공식

데이터가 **크기 순서대로 정렬(Sorted)**되어 있을 때, 정가운데(Middle) 값을 찔러보며 탐색 범위를 매번 절반(1/2)씩 날려버리는 초고속 탐색 방법입니다.

💡 왜 100개의 데이터도 단 7번 만에 찾을 수 있을까?

- 1번 질문: 100개 ➔ 50개로 감소
- 2번 질문: 50개 ➔ 25개로 감소
- 3번 질문: 25개 ➔ 12개로 감소
- 4번 질문: 12개 ➔ 6개로 감소
- 5번 질문: 6개 ➔ 3개로 감소
- 6번 질문: 3개 ➔ 1개로 감소
- 7번 질문: 1개 ➔ 정답 확인!
- **공식**: 데이터 수가 N개일 때, 최악의 경우 필요한 탐색 횟수 k는 $2^k \geq N$**을 만족하는 가장 작은 정수** k입니다.
    - $N = 100$일 때: $2^6 = 64 < 100 \leq 2^7 = 128$ 이므로 **최대 7번**!
    - $N = 30$일 때: $2^4 = 16 < 30 \leq 2^5 = 32$ 이므로 **최대 5번**!

---

7. 실전 킬러 연습문제

[문제 1] 버블 정렬 회전 상태 추적

다음과 같이 5개의 숫자로 구성된 배열이 있습니다.
`[9, 4, 7, 2, 6]`
이 배열을 버블 정렬 알고리즘을 사용하여 오름차순으로 정렬할 때, **2회전(Pass 2)이 완료된 직후의 배열 상태**로 올바른 것은 무엇일까요?

[해설]
직접 손으로 각 회전을 추적합니다.

- **1회전**:
    - (9, 4) 비교 ➔ 바꿈: `[4, 9, 7, 2, 6]`
    - (9, 7) 비교 ➔ 바꿈: `[4, 7, 9, 2, 6]`
    - (9, 2) 비교 ➔ 바꿈: `[4, 7, 2, 9, 6]`
    - (9, 6) 비교 ➔ 바꿈: `[4, 7, 2, 6, 9]`
    - ➔ 1회전 완료 상태: `[4, 7, 2, 6, 9]` (9 고정)
- **2회전**: (확정된 9를 제외한 앞의 4개 비교)
    - (4, 7) 비교 ➔ 그대로: `[4, 7, 2, 6, 9]`
    - (7, 2) 비교 ➔ 바꿈: `[4, 2, 7, 6, 9]`
    - (7, 6) 비교 ➔ 바꿈: `[4, 2, 6, 7, 9]`

- ➔ 2회전 완료 상태: `[4, 2, 6, 7, 9]` (7 고정)

[정답] **[4, 2, 6, 7, 9]**

[문제 2] 선택 정렬의 실제 교환 횟수

배열 `[6, 3, 8, 2, 5]`를 선택 정렬 알고리즘을 사용하여 오름차순으로 정렬하려고 합니다. 단, 어떤 회전에서 선택된 최솟값이 이미 그 회전의 첫 번째 자리에 위치하여 **자기 자신과 자리를 바꾸는 경우는 교환 횟수에 포함하지 않는다**고 할 때, 전체 정렬이 완료될 때까지 **서로 다른 두 숫자가 자리를 맞바꾼 (Swap) 총 횟수**는 몇 회일까요?

[해설]
선택 정렬의 매 회전을 손으로 추적하며 실제 위치가 바뀌는 교환만 카운트합니다.

1. **1회전**: 전체 `[6, 3, 8, 2, 5]` 중 최솟값은 2입니다.
   - 첫 번째 자리의 6과 2를 맞바꿉니다. ➔ **(교환 1회 발생)**
   - 배열 상태: `[2, 3, 8, 6, 5]` (2 고정)
2. **2회전**: 남은 `[3, 8, 6, 5]` 중 최솟값은 3입니다.
   - 3은 이미 두 번째 자리에 있으므로 교환이 발생하지 않습니다. ➔ (교환 0회)
   - 배열 상태: `[2, 3, 8, 6, 5]` (3 고정)
3. **3회전**: 남은 `[8, 6, 5]` 중 최솟값은 5입니다.
   - 세 번째 자리의 8과 5를 맞바꿉니다. ➔ **(교환 1회 발생)**
   - 배열 상태: `[2, 3, 5, 6, 8]` (5 고정)
4. **4회전**: 남은 `[6, 8]` 중 최솟값은 6입니다.
   - 6은 이미 네 번째 자리에 있으므로 교환이 발생하지 않습니다. ➔ (교환 0회)
   - 최종 정렬 완료: `[2, 3, 5, 6, 8]`

따라서 서로 다른 두 숫자가 실제로 교환된 총 횟수는 1회전(1회) + 3회전(1회) = **2회**입니다.

[정답] **2회**

[문제 3] 뒤섞인 번호표의 최소 교환 횟수 퍼즐

어느 책상 위에 5장의 번호표가 다음과 같은 순서로 놓여 있습니다.
`[4, 5, 1, 2, 3]`
한 번에 **임의의 서로 다른 두 장의 카드를 선택하여 자리를 맞바꾸는 조작**만 할 수 있습니다. 이 카드들을 오름차순인 `[1, 2, 3, 4, 5]`로 정렬하기 위해 필요한 **최소 맞바꾸기(Swap) 횟수**는 몇 회일까요?

[해설]
알고리즘의 제한이 없으므로, 한 번 바꿀 때마다 최대한 많은 카드가 제자리를 찾도록 맞바꿉니다.

- 현재 상태: `[4, 5, 1, 2, 3]` (목표: 1번째 자리에 1, 2번째에 2, 3번째에 3, 4번째에 4, 5번째에 5)

1. 1번 자리에 있는 '4'를 원래 자기 자리인 4번 자리에 있는 '2'와 맞바꿉니다.
    - 결과: `[2, 5, 1, 4, 3]` (4가 제자리를 찾음! 교환 1회)
2. 1번 자리에 있는 '2'를 2번 자리에 있는 '5'와 맞바꿉니다.
    - 결과: `[5, 2, 1, 4, 3]` (2가 제자리를 찾음! 교환 2회)
3. 1번 자리에 있는 '5'를 5번 자리에 있는 '3'과 맞바꿉니다.
    - 결과: `[3, 2, 1, 4, 5]` (5가 제자리를 찾음! 교환 3회)
4. 마지막으로 1번 자리에 있는 '3'과 3번 자리에 있는 '1'을 맞바꿉니다.
    - 결과: `[1, 2, 3, 4, 5]` (1과 3이 동시에 제자리를 찾으며 정렬 완료! 교환 4회)

모든 카드가 제자리가 아니었으며(5장), 5장이 하나의 순환 고리를 이루고 있었으므로 최소 교환 횟수는 $5 - 1 = 4$회입니다.

[정답] **4회**

[문제 4] 사전에서 단어 찾기 (이진 탐색 비교 횟수)

알파벳 순서로 완벽하게 정렬되어 있는 60페이지 분량의 영단어 사전이 있습니다. 각 페이지에는 1개의 대표 단어가 적혀 있습니다. 어떤 학생이 이진 탐색(Binary Search) 알고리즘을 사용하여 특정한 단어를 찾으려고 합니다.
원하는 단어가 사전에 반드시 존재한다고 할 때, 이 학생이 그 단어를 찾기 위해 사전을 펼쳐서 확인해야 하는 **최대 페이지 확인 횟수(최악의 경우 비교 횟수)**는 몇 회일까요?

[해설]
이진 탐색은 한 번 확인할 때마다 탐색 범위가 절반으로 줄어듭니다.

- 전체 데이터 수 $N = 60$입니다.
- 최대 확인 횟수 k는 $2^k \geq 60$을 만족하는 최소의 정수입니다.
    - $2^1 = 2$
    - $2^2 = 4$
    - $2^3 = 8$
    - $2^4 = 16$
    - $2^5 = 32 < 60$
    - $2^6 = 64 \geq 60$
- $2^5(32)$로는 60개의 페이지를 모두 판별할 수 없지만, $2^6(64)$이면 60개의 페이지 중 어느 위치에 있든 최대 6번 만에 무조건 해당 단어를 찾아낼 수 있습니다.

[정답] **6회**

🔌 제3특강: 디지털 논리 회로 및 컴퓨터 가산기 특강

> **"컴퓨터의 뇌(CPU) 속에서는 0과 1이 전깃줄을 타고 흐릅니다. 기본개념에서 배운 논리 게이트들이 복잡하게 연결된 회로를 만났을 때, 전선을 따라 신호를 추적하고 컴퓨터 덧셈기의 원리를 정복해 봅시다!"**

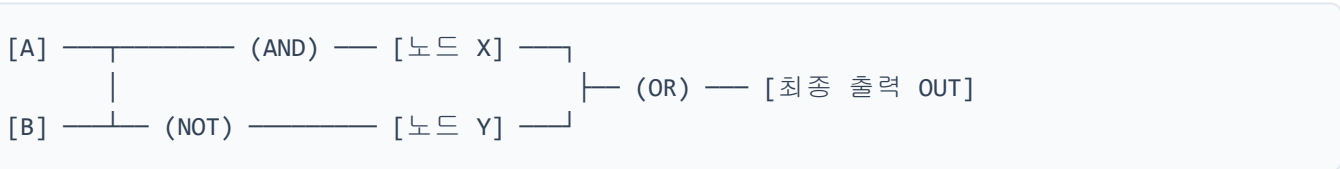
디미고 소질·적성검사의 컴퓨팅 사고력 영역에서는 하드웨어의 근본 동작 원리인 **복합 논리 회로(Composite Logic Circuit)**와 컴퓨터가 2진수 덧셈을 수행하는 핵심 장치인 **가산기(Adder)**에 관한 문제가 출제됩니다.

이 특강에서는 기본개념에서 익힌 논리 게이트들을 바탕으로, 시험지에 그려진 다단 회로를 1초의 망설임 없이 손으로 추적하는 방법과 **반가산기(Half Adder)** 및 **전가산기(Full Adder)**의 연결 구조를 완벽하게 마스터합니다.

---

1. 복합 논리 회로 다이어그램과 3단계 신호 추적법

시험지에 여러 개의 게이트가 거미줄처럼 얽혀 있는 복합 회로가 나오더라도 전혀 당황할 필요가 없습니다.



💡 시험장에서 절대 틀리지 않는 3단계 행동 강령

1. **1단계: 입력단에 숫자 크게 적기 ➔** 입력단(A, B, C 등)에 주어진 0 또는 1을 연필로 전선 시작점에 크게 적습니다.
2. **2단계: 분기점 신호 복사하기 ➔** 전선이 갈라지는 분기점(검은 점 또는 교차점)에서는 동일한 0 또는 1의 신호가 양쪽 갈래 전선으로 그대로 복사되어 흘러갑니다. 갈라진 전선 위에도 숫자를 각각 적어둡니다.
3. **3단계: 중간 노드 값 기록하기 ➔** 각 게이트를 통과할 때마다 게이트 바로 뒤쪽 전선 위에 중간 결과값(노드 X, Y 등)을 적어놓습니다. 앞 단계의 계산 결과를 바탕으로 마지막 게이트까지 신호를 전달하면 최종 출력이 100% 정확하게 도출됩니다.

---

2. 신호 반전과 만능 게이트: NAND와 NOR 회로 분석

기본 4대 게이트(AND, OR, NOT, XOR) 외에, 게이트 끝에 작은 동그라미(NOT)가 붙어 출력을 반대로 뒤집는 반전 게이트들이 자주 출제됩니다.

💡 NAND 게이트 (NOT + AND)

- **기호**: AND 게이트(D 모양) 출력단 끝에 **작은 동그라미(O)**가 붙어 있습니다.
- **동작 규칙**: AND 연산 결과를 그대로 반대로 뒤집습니다.
    - 입력이 둘 다 1일 때만 출력은 **0**이 되고, 그 외에는 항상 **1**을 출력합니다.

💡 NOR 게이트 (NOT + OR)

- **기호**: OR 게이트(화살촉 모양) 출력단 끝에 **작은 동그라미(O)**가 붙어 있습니다.
- **동작 규칙**: OR 연산 결과를 그대로 반대로 뒤집습니다.
    - 입력 중 하나라도 1이면 출력은 **0**이 되고, 둘 다 0일 때만 **1**을 출력합니다.

> 📌 **상식 돋보기: 왜 반도체에서는 NAND를 좋아할까?**
> NAND 게이트는 **만능 게이트(Universal Gate)**라고 불립니다. 놀랍게도 NAND 게이트들만 적절히 연결하면 NOT, AND, OR, XOR 등 세상의 모든 논리 회로를 전부 만들어낼 수 있기 때문에, 실제 컴퓨터 메모리(NAND 플래시 메모리) 제조에 가장 핵심적으로 쓰입니다!

---

3. 컴퓨터의 2진수 덧셈기 1단계: 반가산기 (Half Adder)

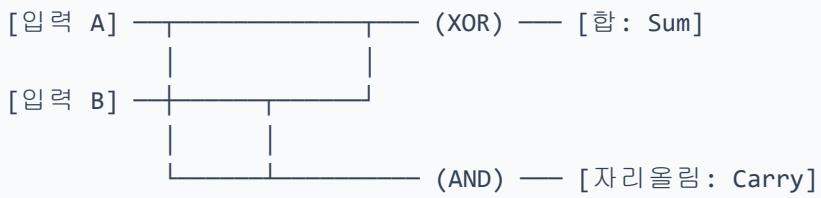
컴퓨터가 어떻게 0과 1만으로 $1+1=2$라는 덧셈을 수행하는지 그 비밀을 담고 있는 회로입니다.

💡 1비트 2진수 덧셈의 4가지 경우

- $0+0=0$ ➔ **자리올림(Carry) 0, 합(Sum) 0**
- $0+1=1$ ➔ **자리올림(Carry) 0, 합(Sum) 1**
- $1+0=1$ ➔ **자리올림(Carry) 0, 합(Sum) 1**
- $1+1=10_{(2)}$ ➔ **자리올림(Carry) 1, 합(Sum) 0**

🎯 게이트와의 1:1 매칭 발견!

- **합(Sum)의 결과 (0, 1, 1, 0)**:
    - 입력 A와 B가 **서로 다를 때만 1**이 됩니다 ➔ **XOR 게이트**!
- **자리올림(Carry)의 결과 (0, 0, 0, 1)**:
    - 입력 A와 B가 **둘 다 1일 때만 1**이 됩니다 ➔ **AND 게이트**!



- **반가산기(Half Adder)의 정의**:
  - 1비트짜리 2진수 2개(A, B)를 더하여 **합(Sum = $A \oplus B$)**과 **자리올림(Carry = $A \cdot B$)**을 만들어 내는 회로입니다.
  - 아랫자리에서 올라오는 올림수를 받아들이지 못하고 자기들끼리만 더하기 때문에 '절반짜리 가산기(Half Adder)'라고 부릅니다.

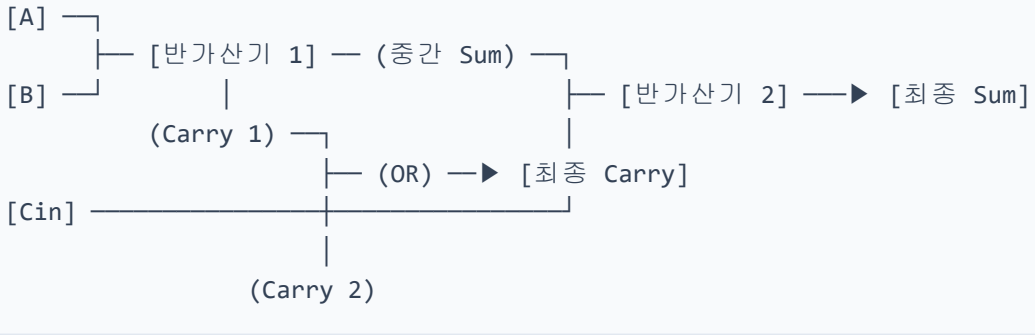
---

4. 컴퓨터의 2진수 덧셈기 2단계: 전가산기 (Full Adder) 구조 맛보기

실제 여러 자릿수의 덧셈을 하려면, 아랫자리에서 올라온 **'받아올림수(Carry In, C_{in})'**까지 포함해서 **총 3개의 1비트**를 한꺼번에 더할 수 있어야 합니다.

💡 전가산기의 블록 구조

전가산기는 복잡한 새 부품을 쓰는 것이 아니라, **반가산기 2개와 OR 게이트 1개**를 연결하여 완성합니다!



- **전가산기의 계산 규칙 (3개 비트 더하기: $A + B + C_{in}$)**:
  - 1이 0개일 때: $0+0+0=0$ ➔ **Sum = 0, Carry = 0**
  - 1이 1개일 때: $1+0+0=1$ ➔ **Sum = 1, Carry = 0**
  - 1이 2개일 때: $1+1+0=10_{(2)}$ ➔ **Sum = 0, Carry = 1**
  - 1이 3개일 때: $1+1+1=11_{(2)}$ ➔ **Sum = 1, Carry = 1** (최고 빈출!)

---

5. 실전 킬러 연습문제

[문제 1] 3단 복합 논리 회로의 신호 추적

다음과 같이 연결된 논리 회로가 있습니다.

- 세 개의 입력 신호는 각각 $A=1, B=0, C=1$ 입니다.
- 입력 A와 B가 **XOR 게이트**로 들어갑니다. (출력 X)
- 입력 B가 **NOT 게이트**를 통과합니다. (출력 Y)
- 신호 X와 신호 Y가 **AND 게이트**로 들어갑니다. (출력 Z)
- 마지막으로 신호 Z와 입력 C가 **OR 게이트**로 들어가 최종 출력 OUT을 만듭니다.

최종 출력값 OUT은 0과 1 중 무엇일까요?

[해설]
3단계 신호 추적법에 따라 각 노드의 신호를 순서대로 구합니다.

1. 입력: $A=1, B=0, C=1$
2. 노드 $X = A \text{ XOR } B$:
    - $1 \text{ XOR } 0 = 1$ (두 입력이 서로 다르므로 1)
3. 노드 $Y = \text{NOT } B$:
    - $\text{NOT } 0 = 1$
4. 노드 $Z = X \text{ AND } Y$:
    - $1 \text{ AND } 1 = 1$ (둘 다 1이므로 1)
5. 최종 $OUT = Z \text{ OR } C$:
    - $1 \text{ OR } 1 = 1$

[정답] **1**

[문제 2] NAND 게이트 결합 회로 분석

어느 회로에 두 입력 $A=1, B=1$이 들어옵니다.

- 두 입력 A와 B가 먼저 **NAND 게이트**로 들어갑니다. (출력 P)
- 이 출력 P의 전선이 두 갈래로 나뉘어, 하나의 **NAND 게이트**의 두 입력단에 동시에 들어갑니다. (출력 Q)

최종 출력 Q의 값은 얼마일까요?

[해설]

1. 1단계: $P = A \text{ NAND } B$
    - $A=1, B=1$ 일 때, AND 결과는 1입니다.
    - NAND는 AND의 반대이므로 출력 P = **0** 입니다.
2. 2단계: 출력 $P(0)$가 두 갈래로 나뉘어 다음 NAND 게이트의 두 입력으로 동시에 들어갑니다.
    - 즉, 입력은 $(0, 0)$입니다.
    - $0 \text{ AND } 0 = 0$ 이므로, 이를 뒤집은 NAND의 최종 출력은 Q = **1** 입니다.
3. 💡 **보너스 통찰**: NAND 게이트의 두 입력을 하나로 묶으면 신호가 반전되는 **NOT 게이트**의 역할을 하게 됩니다! $(1 \rightarrow 0 \rightarrow 1)$

[정답] **1**

[문제 3] 반가산기(Half Adder)의 입출력 신호 계산

반가산기 회로에 입력 $A = 1, B = 1$을 넣었을 때 생성되는 합(Sum)과 자리올림(Carry)의 값으로 올바른 묶음은 무엇일까요?

[해설]
반가산기는 2진수 덧셈 $1 + 1 = 10_{(2)}$을 수행합니다.

- 합(Sum) 출력:
  - $\text{Sum} = A \text{ XOR } B = 1 \text{ XOR } 1 = \mathbf{0}$
- 자리올림(Carry) 출력:
  - $\text{Carry} = A \text{ AND } B = 1 \text{ AND } 1 = \mathbf{1}$
- 따라서 합은 0, 올림수는 1입니다.

[정답] **Sum = 0, Carry = 1**

[문제 4] 전가산기(Full Adder)의 3비트 덧셈

어느 전가산기(Full Adder)의 세 입력단에 다음과 같은 신호가 입력되었습니다.

- 첫 번째 비트 $A = 1$
- 두 번째 비트 $B = 1$
- 아랫자리에서 올라온 자리올림수 $C_{in} = 1$

이 전가산기가 덧셈을 완료한 후 내보내는 **최종 합(Sum)**과 **최종 자리올림(Carry)**의 값은 각각 얼마일까요?

[해설]
전가산기는 3개의 1비트 숫자들의 덧셈($A + B + C_{in}$)을 수행합니다.

- 10진수 덧셈: $1 + 1 + 1 = 3$
- 이를 2진수로 변환하면 $3 = 11_{(2)}$ 입니다.
- 따라서:
  - 1의 자리에 해당하는 **최종 합(Sum) = 1**
  - 2의 자리에 해당하는 **최종 자리올림(Carry) = 1**
- 3개의 입력이 모두 1일 때, 합과 자리올림 모두 1이 출력됩니다.

[정답] **Sum = 1, Carry = 1**